1. SQLite Always Validates Its Inputs

SQLite should never crash, overflow a buffer, leak memory, or exhibit any other harmful behavior, even when presented with maliciously malformed SQL inputs or database files. SQLite should always detect erroneous inputs and raise an error, not crash or corrupt memory. Any malfunction caused by an SQL input or database file is considered a serious bug and will be promptly addressed when brought to the attention of the SQLite developers. SQLite is extensively fuzz-tested to help ensure that it is resistant to these kinds of errors.

Nevertheless, bugs happen. If you are writing an application that sends untrusted SQL inputs or database files to SQLite, there are additional steps you can take to help reduce the attack surface and prevent zero-day exploits caused by undetected bugs.

1.1. Untrusted SQL Inputs

Applications that accept untrusted SQL inputs should take the following precautions:

1. Set the SQLITE_DBCONFIG_DEFENSIVE flag. This prevents ordinary SQL statements from deliberately corrupting the database file. SQLite should be proof against attacks that involve both malicious SQL inputs and a maliciously corrupted database file at the same time. Nevertheless, denying a script-only attacker access to corrupt database inputs provides an extra layer of defense.

2. Reduce the limits that SQLite imposes on inputs. This can help prevent denial of service attacks and other kinds of mischief that can occur as a result of unusually large inputs. You can do this either at compile-time using -DSQLITE_MAX_... options, or at run-time using the sqlite3_limit() interface. Most applications can reduce limits dramatically without impacting functionality. The table below provides some suggestions, though exact values will vary depending on the application:

Limit Setting	Default Value	High-security Value
LIMIT_LENGTH	1,000,000,000	1,000,000

LIMIT_SQL_LENGTH	1,000,000,000	100,000
LIMIT_COLUMN	2,000	100
LIMIT_EXPR_DEPTH	1,000	10
LIMIT_PARSER_DEPTH	2,500	100
LIMIT_COMPOUND_SELECT	500	3
LIMIT_VDBE_OP	250,000,000	25,000
LIMIT_FUNCTION_ARG	127	8
LIMIT_ATTACH	10	0
LIMIT_LIKE_PATTERN_LENGTH	50,000	50
LIMIT_VARIABLE_NUMBER	999	10
LIMIT_TRIGGER_DEPTH	1,000	10

Large limits might enable an attacker who is able to inject arbitrary SQL to construct a query that will cause deep recursion and hence a CPU stack overflow. The default limits are sufficient to prevent this on most machines, but if running on a processor with limited stack space, take care to set LIMIT_EXPR_DEPTH and LIMIT_TRIGGER_DEPTH to small values, such as 10.

3. Consider using the sqlite3_set_authorizer() interface to limit the scope of SQL that will be processed. For example, an application that does not need to change the database schema might add an sqlite3_set_authorizer() callback that causes any CREATE or DROP statement to fail.

4. The SQL language is very powerful, and so it is always possible for malicious SQL inputs (or erroneous SQL inputs caused by an application bug) to submit SQL that runs for a very long time. To prevent this from becoming a denial-of-service attack, consider using the sqlite3_progress_handler() interface to invoke a callback periodically as each SQL statement runs, and have that callback return non-zero to abort the statement if the statement runs for too long. Alternatively, set a timer in a separate thread and invoke sqlite3_interrupt() when the timer goes off to prevent the SQL statement from running forever.

5. Limit the maximum amount of memory that SQLite will allocate using the sqlite3_hard_heap_limit64() interface. This helps prevent denial-of-service attacks. To find out how much heap space an application actually needs, run the it against typical inputs and then measure the maximum instantaneous memory usage with the sqlite3_memory_highwater() interface. Set the hard heap limit to the maximum observed instantaneous memory usage plus some margin.

6. Consider setting the SQLITE_MAX_ALLOCATION_SIZE compile-time option to something smaller than its default value of 2147483391 (0x7fffeff). A value of 100000000 (100 million) or even smaller would not be unreasonable, depending on the application.

7. For embedded systems, consider compiling SQLite with the -DSQLITE_ENABLE_MEMSYS5 option and then providing SQLite with a fixed chunk of memory to use as its heap via the sqlite3_config(SQLITE_CONFIG_HEAP) interface. This will prevent malicious SQL from executing a denial-of-service attack by using an excessive amount of memory. If (say) 5 MB of memory is provided for SQLite to use, once that much has been consumed, SQLite will start returning SQLITE_NOMEM errors rather than soaking up memory needed by other parts of the application. This also sandboxes SQLite's memory so that a write-after-free error in some other part of the application will not cause problems for SQLite, or vice versa.

8. To control memory usage in the printf() SQL function, compile with "-DSQLITE_PRINTF_PRECISION_LIMIT=100000" or some similarly reasonable value. This #define limits the width and precision for %-substitutions in the printf() function, and thus prevents a hostile SQL statement from consuming large amounts of RAM via constructs such as `"printf('%1000000000s','hi')"`.

Note that SQLite uses its built-in printf() internally to help it format the sql column in the sqlite_schema table. For that reason, no table, index, view, or trigger definition can be much larger than the precision limit. You can set a precision limit of less than 100000, but be careful that whatever precision limit you use is at least as long as the longest CREATE statement in your schema.

1.2. Untrusted SQLite Database Files

Applications that read or write SQLite database files of uncertain provenance should take precautions enumerated below.

Even if the application does not deliberately accept database files from untrusted sources, beware of attacks in which a local database file is altered. For best security, any database file which might have ever been writable by an agent in a different security domain should be treated as suspect.

9. If the application includes any custom SQL functions or custom virtual tables that have side effects or that might leak privileged information, then the application should use one or more of the techniques below to prevent a maliciously crafted database schema from surreptitiously running those SQL functions and/or virtual tables for nefarious purposes:

    a. Invoke sqlite3_db_config(db,SQLITE_DBCONFIG_TRUSTED_SCHEMA,0,0) on each database connection as soon as it is opened.
    b. Run the PRAGMA trusted_schema=OFF statement on each database connection as soon as it is opened.
    c. Compile SQLite using the -DSQLITE_TRUSTED_SCHEMA=0 compile-time option.
    d. Disable the surreptitious use of custom SQL functions and virtual tables by setting the SQLITE_DIRECTONLY flag on all custom SQL functions and the SQLITE_VTAB_DIRECTONLY flag on all custom virtual tables.
    e. Ensure that custom virtual tables that run SQL statements based on arguments to the CREATE VIRTUAL TABLE statement in the database schema use sqlite3_prepare_v3() with the SQLITE_PREPARE_FROM_DDL option to prevent bypass of the SQLITE_DBCONFIG_TRUSTED_SCHEMA setting in item (9a) above.

10. If the application does not use triggers or views, consider disabling the unused capabilities with:

```
sqlite3_db_config(db,SQLITE_DBCONFIG_ENABLE_TRIGGER,0,0);
sqlite3_db_config(db,SQLITE_DBCONFIG_ENABLE_VIEW,0,0);
```

If an application is particularly security sensitive, then the following extra layers of defense might be justified. These extra defenses come with performance costs, however, and so may not be appropriate in every situation:

11. Run PRAGMA integrity_check or PRAGMA quick_check on the database as the first SQL statement after opening the database files and prior to running any other SQL statements. Reject and refuse to process any database file containing errors.

12. Enable the PRAGMA cell_size_check=ON setting.

13. Do not enable memory-mapped I/O. In other words, make sure that PRAGMA mmap_size=0.

2. Summary

The precautions above are not required in order to use SQLite safely with potentially hostile inputs. However, they do provide an extra layer of defense against zero-day exploits and are encouraged for applications that pass data from untrusted sources into SQLite.

This page was last updated on 2026-07-10 16:44:46Z